Silvestre Ronin, silver3@rpi.edu, 662043925
Kurbiel Chris, kurbic@rpi.edu, 662041666
Homework 3
Professor Radev
COGNITIVE MODELING - COGS 4210 - 01
March 21, 2026

Good evening Professor,

I sincerely apologise for my submission being a couple hours late. I had 2 other assignments due earlier today and a test as well. I had started ahead of time but underestimated how long q2 would take me, and I didn't want to rush the homework. I will be more mindful of the time and start hw4 earlier.

Thank you. Ronin

1 Problem 1 [FINISHED]

All statements that are false: 2, 3, 5, 6, 8.
Assumption: In statement 4, we consider "clever numerical approximation" as said in the slides to mean that we can indeed evaluate numerically. Thus, this statement is true.

2 Problem 2 [FINISHED]

The code for this problem can be found here.

(i) Output for hyperparameter tuning on $v \in [0.5, 1.5]$ for a total of 25 different drift rates:

```
diff_mean
✓ 0.0s

array([0.02013907, 0.0193703 , 0.01796282, 0.02162733, 0.01446707,
       0.01920748, 0.01710024, 0.02127788, 0.01959318, 0.01676891,
       0.01652615, 0.01971371, 0.01684492, 0.01646245, 0.01868264,
       0.01291996, 0.01933156, 0.0209211 , 0.02017255, 0.02146621,
       0.01393765, 0.01726876, 0.01547631, 0.01741579, 0.01664844])
```

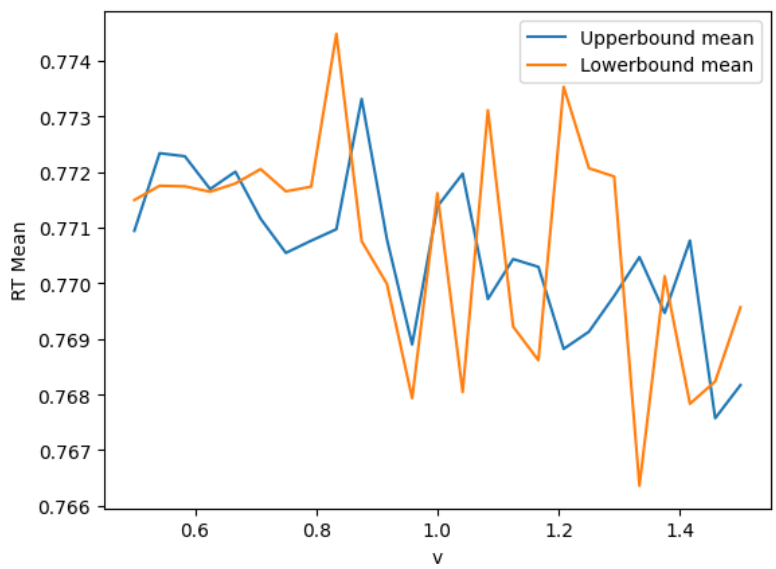
ANALYSIS

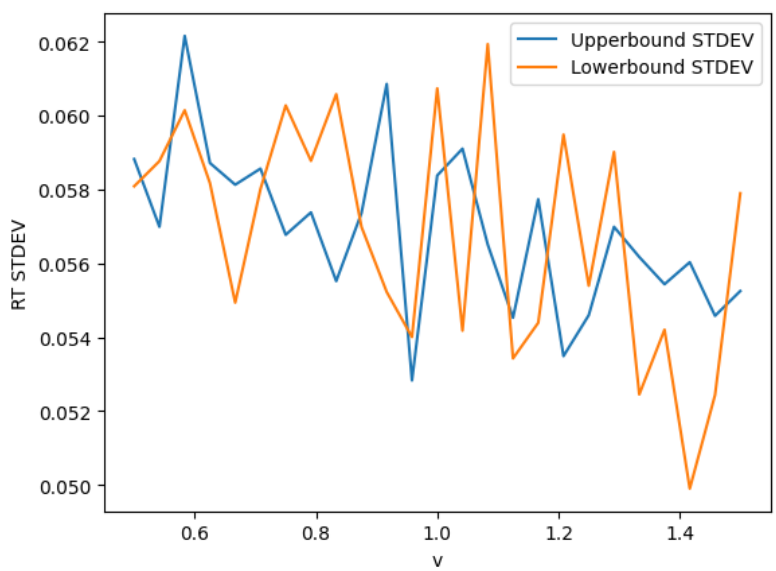
The constant terms were $a = 0.5, \beta = 0.5, \tau = 0.7, N = 2000$. From our results, we observe that the difference between the mean response times for the upper and lower boundaries were consistently quite small, around values of 0.015 - 0.02, which was consistent across all 25 different drift values tested. Our difference equation was upperbound_mean$_i$ − lowerbound_mean$_i$, thus indicating that consistently, the upperbound mean was a bit greater than the lowerbound mean. This is counterintuitive since one may expect that the drift value may have a strong impact on the difference in timing between the correct and error responses. However, the data suggests that the overall impact of the drift value is limited since changing drift value does not shown any patterns in how the difference of means changes.

(ii)

The arrays of the means and STDEVs are not very interesting to look at but are included in the notebook for completeness. The outputs shown here are graphs, which provide a much easier analytical perspective.

(a) Hyperparameter tuning on $v \in [0.5, 1.5]$



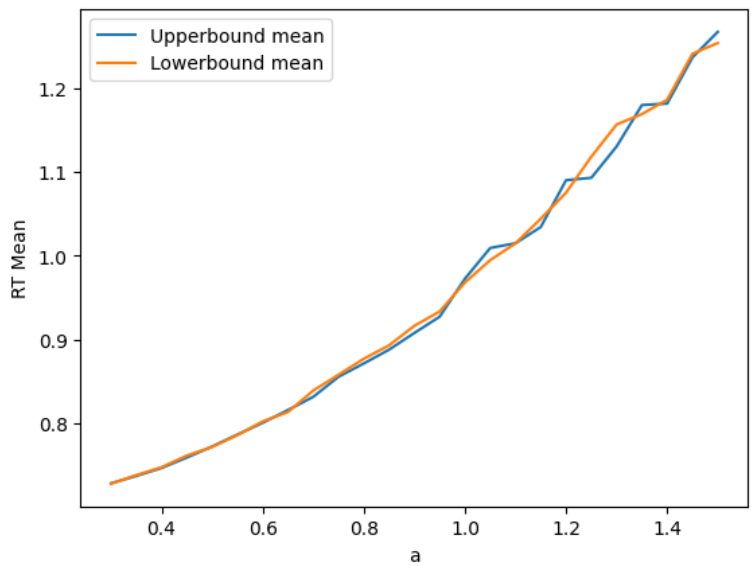


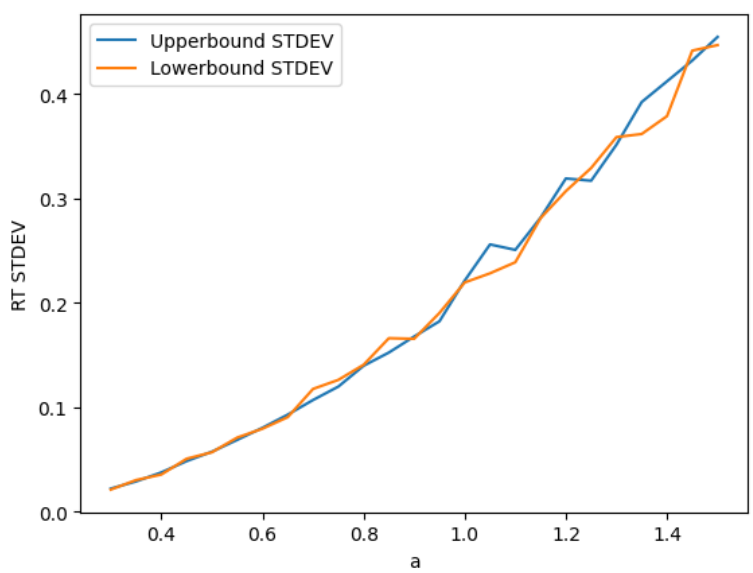
ANALYSIS

When varying v, we observe not strong patterns in both the RT Mean and STDEV graphs. The RT means stay close to each other for both the upper and lower bounds, suggesting that the drift factor impacts both boundaries in the same way. There is a small, but noticeable decrease in the RT Mean for both boundaries as we increase v,

which may indicate that higher drift leads to faster decisions in general. With the RT STDEV graph, there are even less trends, as STDEV stays rather the same throughout values of v.

(b) Hyperparameter tuning on $a \in [0.3, 1.5]$



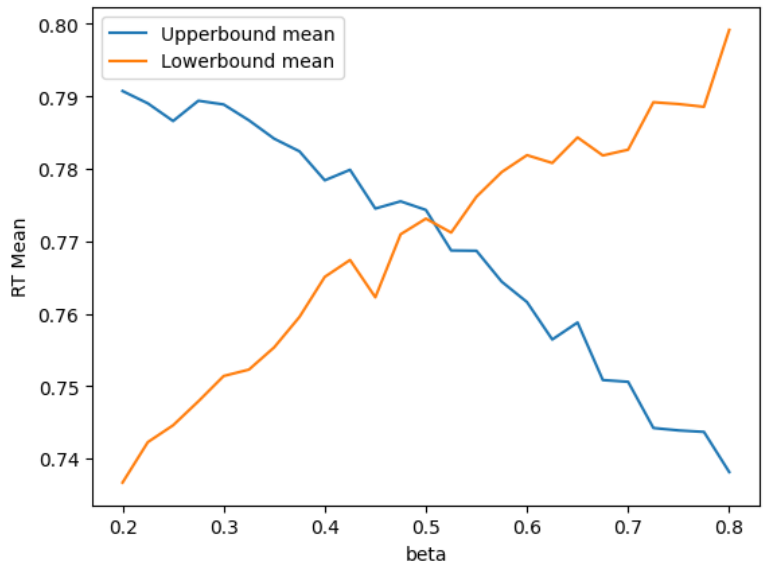


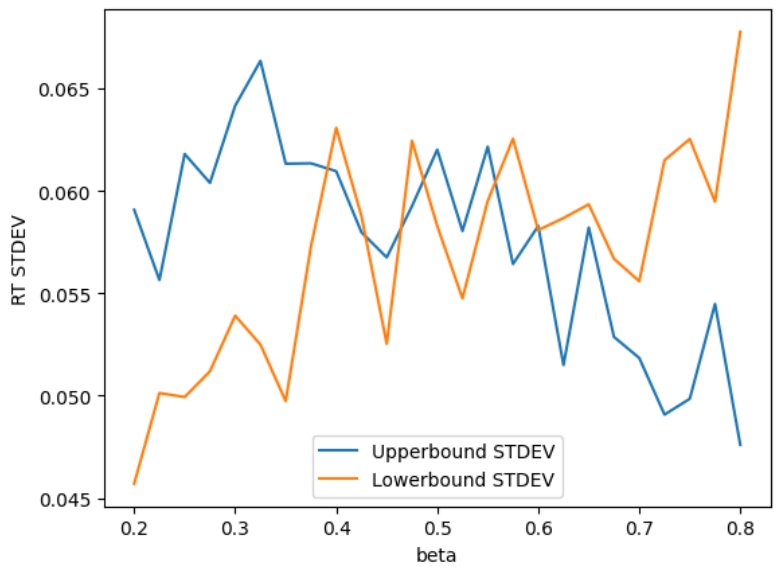
ANALYSIS

When varying a, there is much more noticeable patterns in both graphs. As we increase a, the means of both upper and lower boundaries both increase in similar fashion, which suggests that larger boundary separations results in more time to make decisions. With the STDEV graph, we observe a similar pattern, in which the STDEV of both boundaries increase as a increases. This suggests that as the boundary separation

grows larger, there is more variability in decision time.

(c) Hyperparameter tuning on $\beta \in [0.2, 0.8]$



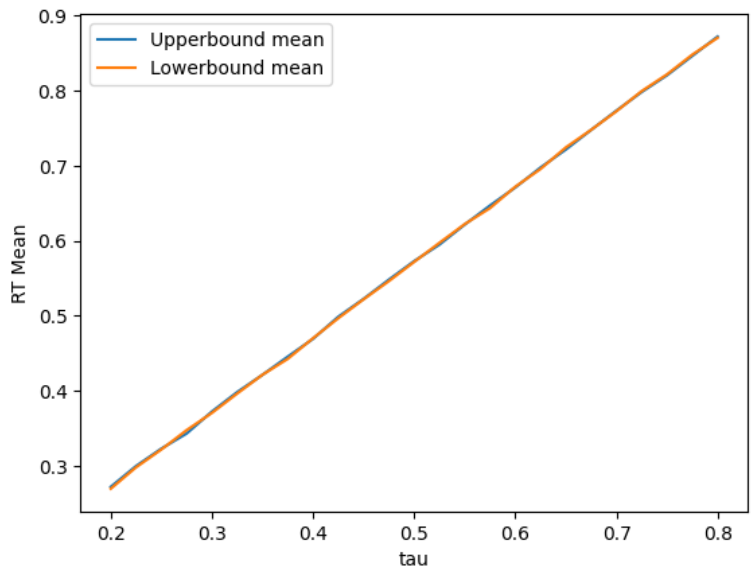


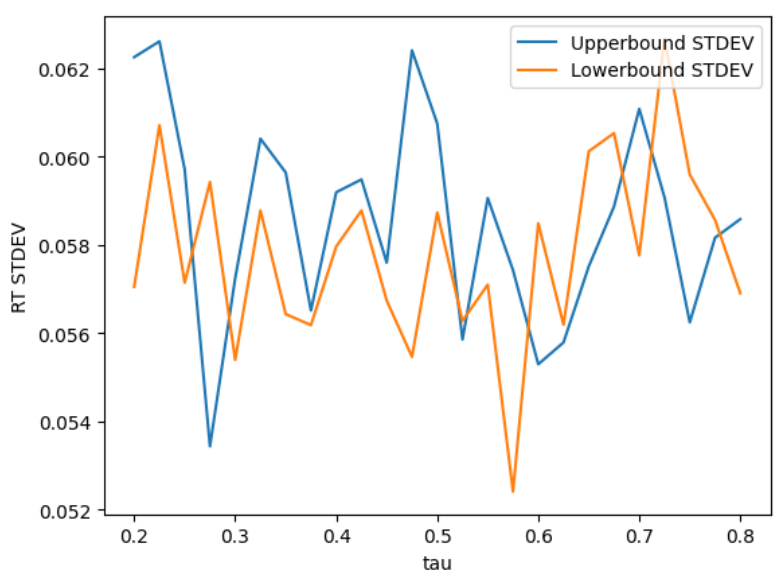
ANALYSIS

When varying β, we see a clear pattern in the RT Mean graph. As β increases, so does the lowerbound mean, while on the other hand, the upperbound mean decreases. This suggests that as we move our starting point closer to the upper boundary, it will take less time to make decisions that result in that upper boundary. The opposite holds true, in which if we start closer to the upper boundary, it would take more time to get to the lower boundary. This overall explains the asymmetry between the RT mean trends for both boundaries. For the RT STDEV graph, the general trends are

the opposite of the RT mean graph. We observe that the upperbound's STDEV tends to decrease as β increases, and the lowerbound's STDEV increase as β increases. This indicates that as we start closer to a boundary, the reaction times are faster as well as more consistent and have less variation.

(d) Hyperparameter tuning on $\tau \in [0.2, 0.8]$





ANALYSIS

When varying τ, we observe a clear linear relationship between τ and the RT means of both boundaries. As we increase τ, the RT means of both the upper and lower boundaries increase at the same rate as τ. This suggests that τ affects both boundaries equally and that τ adds a constant delay to all response times. Looking at the

RT STDEV graph, there is no discernible trends for either boundary, indicating that variation is not affected by τ.

3 Problem 3 [FINISHED]

We will show that the following identity holds for any given prior and posterior:

$$\mathrm{Var}[\theta] = \mathbb{E}[\mathrm{Var}[\theta \mid y]] + \mathrm{Var}[\mathbb{E}[\theta \mid y]] \tag{1}$$

By definition of variance, we have that $\mathrm{Var}[X] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$. Additionally, from property (1.8) of Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). Bayesian data analysis. CRC press (the textbook listed in the slide reference from week4), we have that $\mathbb{E}[\theta] = \mathbb{E}[\mathbb{E}[\theta \mid y]]$. Now, using these proprties, we expand the right hand side of Equation 1 as follows.

$$\begin{aligned}
\mathbb{E}[\mathrm{Var}[\theta \mid y]] + \mathrm{Var}[\mathbb{E}[\theta \mid y]] &= \mathbb{E}[\mathbb{E}[\theta^2 \mid y] - (\mathbb{E}[\theta \mid y])^2] + \mathbb{E}[(\mathbb{E}[\theta \mid y])^2] - (\mathbb{E}[\mathbb{E}[\theta \mid y]])^2 \\
&= \mathbb{E}[\theta^2] - \mathbb{E}[(\mathbb{E}[\theta \mid y])^2] + \mathbb{E}[(\mathbb{E}[\theta \mid y])^2] - (\mathbb{E}[\theta])^2 \\
&= \mathbb{E}[\theta^2] - (\mathbb{E}[\theta])^2 \\
&= \mathrm{Var}[\theta].
\end{aligned}$$

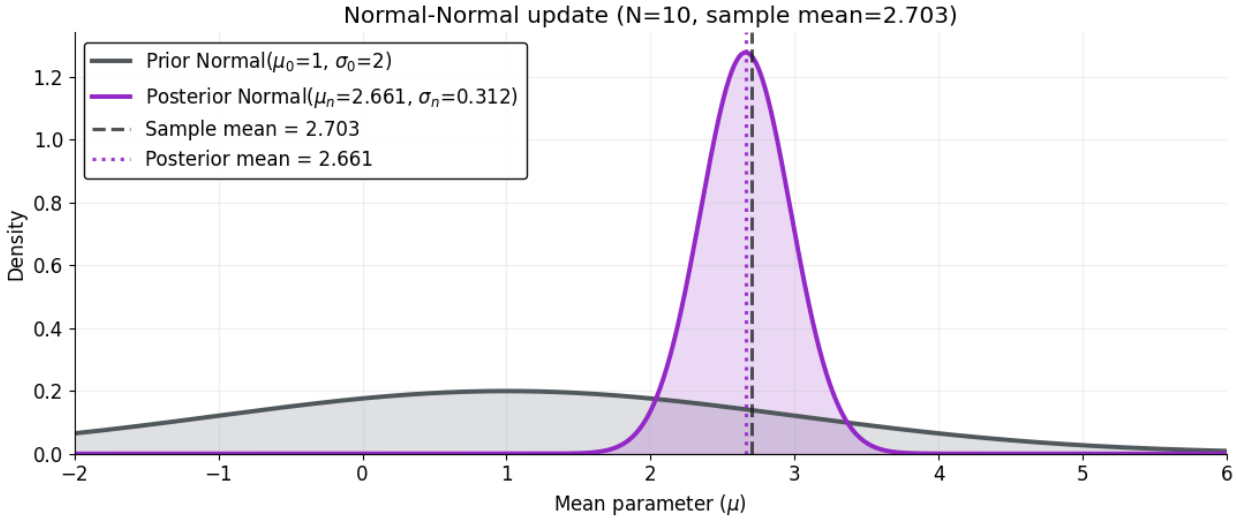
Thus, we have shown that $\mathrm{Var}[\theta] = \mathbb{E}[\mathrm{Var}[\theta \mid y]] + \mathrm{Var}[\mathbb{E}[\theta \mid y]]$.

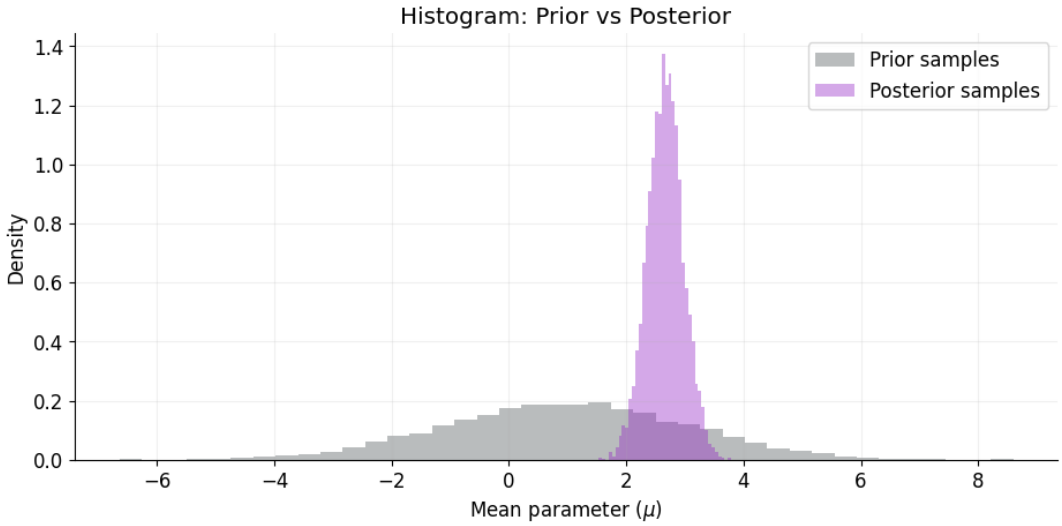
4 Problem 4 [FINISHED]

The code for this problem can be found here.

An entertaining use case for this model is detailed as follows: As Spring season is around the corner, we are curious to record the average number of sneezes per class. Our prior belief was that the number of sneezes is 1 per class. However, the data from our last 10 classes indicates that we have undershot the actual number of sneezes per class.

The outputs of the code are as follows:





5 Problem 5 [FINISHED]

The notebook file for this problem can be found here. Additionally, the stan file for this problem can be found here.

TRUE VALUES

```python
# simulating the data...
N = 100
alpha = 2.3
sigma = 2.
slope = 4.
x = np.random.normal(size=N)
y = alpha + slope * x + sigma * np.random.normal(size=N)

# creating Stan-friendly data dictionary...
data_dict = {
    "N": N,
    "x": x,
    "y": y
}
```

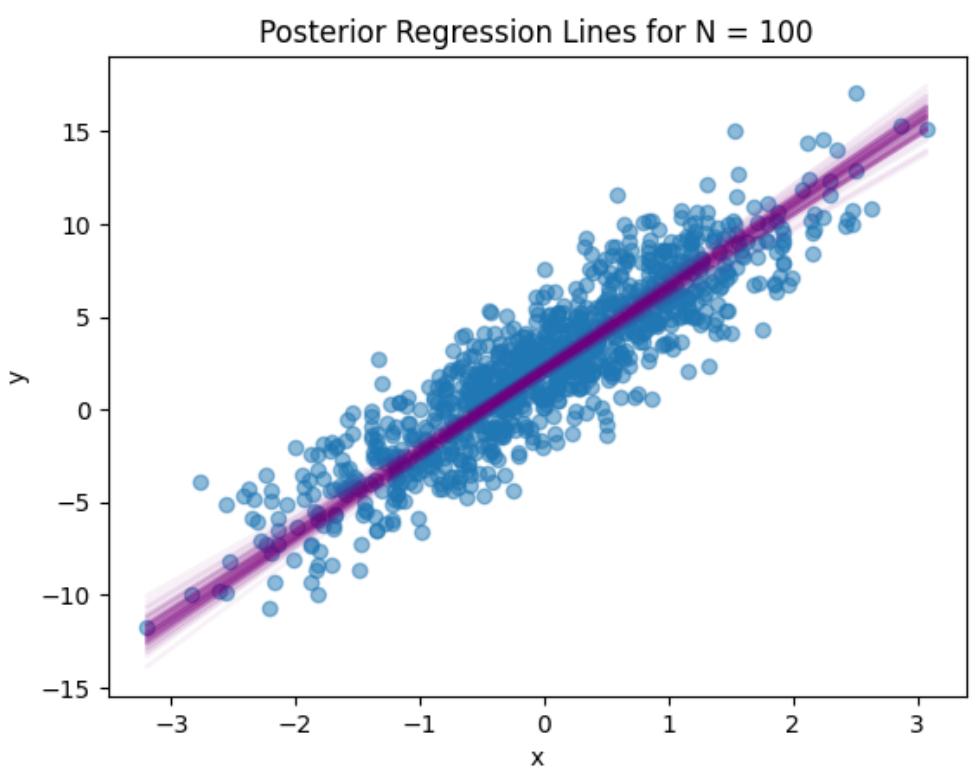
✓ 0.0s

OUTPUT FOR N = 100

```python
fit.summary()
```

[8] ✓ 0.8s

	Mean	MCSE	StdDev	MAD	5%	50%	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	-221.06500	0.013220	1.253700	1.001540	-223.57000	-220.73300	-219.71900	9763.57	12828.6	11834.6	1.000300
alpha	2.16020	0.001561	0.203252	0.201203	1.82802	2.15998	2.49396	17044.20	13885.5	20659.6	1.000080
beta	4.41439	0.001476	0.205950	0.206518	4.07751	4.41422	4.75101	19505.60	13335.6	23643.1	0.999961
sigma2	4.11532	0.004353	0.599369	0.578202	3.24835	4.04928	5.17904	19819.40	14820.9	24023.5	1.000360
sigma	2.02338	0.001049	0.145775	0.143882	1.80232	2.01228	2.27575	19819.40	14820.9	24023.5	1.000380



ANALYSIS FOR N = 100

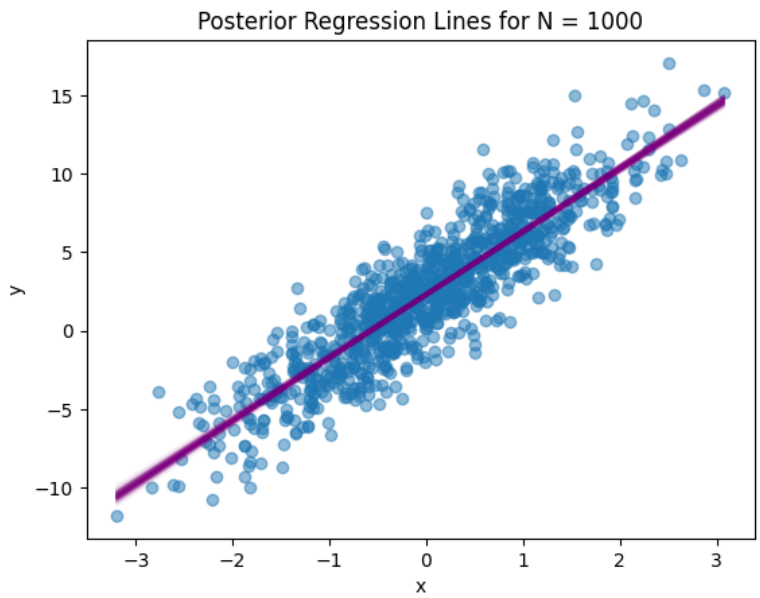
From the above numerical and graphical outputs, we observe that the posterior means are quite close to the actual values of α, σ, β that were used to simulate the data. Specifically, we observe that the estimated posterior means are roughly $\alpha = 2.16, \sigma = 2.02, \beta = 4.41$, which are indeed quite close to the actual values of these parameters. We observe that for each variable, our estimates overshot by a bit. The standard deviations are roughly .20 for α, .15 for σ, and .21 for β, which indicate that there is still a decent amount of uncertainty in the estimates for each parameter. Furthermore, this is visible from the graph, we observe that there is still a noticeable spread in the estimated posterior means. In terms of efficiency, the effective sample size (ESS) are large for all parameters, roughly 20000, which implies that the Markov chains are producing nearly independent samples and exploring the posterior distribution efficiently. In terms of convergence, it is quite good with $\hat{R} = 1$ roughly across each parameter. Thus, overall the MCMC sampling is both quite efficient and well-converged.

OUTPUT FOR N = 1000

```
fit2.summary()
```

✓ 0.6s

	Mean	MCSE	StdDev	MAD	5%	50%	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	-2142.95000	0.012072	1.226030	0.990377	-2145.35000	-2142.63000	-2141.62000	10213.9	14503.2	3576.30	1.00008
alpha	2.30070	0.000466	0.064708	0.064744	2.19389	2.30080	2.40706	19291.4	14423.1	6754.68	1.00014
beta	4.01613	0.000452	0.063050	0.062933	3.91233	4.01642	4.11981	19524.1	14976.6	6836.15	1.00020
sigma2	4.18284	0.001256	0.188593	0.188309	3.88442	4.17784	4.50157	22771.2	15556.7	7973.11	1.00027
sigma	2.04468	0.000306	0.046057	0.046019	1.97089	2.04398	2.12169	22771.2	15556.7	7973.10	1.00027



ANALYSIS FOR N = 1000

From the above numerical and graphical outputs, we observe that the posterior means are quite close to the actual values of α, σ, β that were used to simulate the data. Specifically, we observe that the estimated posterior means are roughly $\alpha = 2.30, \sigma = 2.04, \beta = 4.02$, which are indeed quite close to the actual values of these parameters. We observe that for each variable, our estimates deviate by a very small amount, noticeably lower than for N = 100. The standard deviations are roughly .065 for α, .046 for σ, and .063 for β, which indicate that there is now a much smaller amount of uncertainty in the estimates for each parameter. Furthermore, this is visible from the graph, where we observe that there is much less spread in the estimated posterior means, with the regression lines appearing tightly clustered. In terms of efficiency, the effective sample size (ESS) are large for all parameters, roughly 15000, which implies that the Markov chains are producing nearly independent samples and exploring the posterior distribution efficiently. In terms of convergence, it is quite good with $\hat{R} = 1$ roughly across each parameter. Thus, overall the MCMC sampling is both quite efficient and well-converged.

6 Problem 6

The code for this problem can be found here. Additionally, the stan file for this problem can be found here.

From the description of the problem, the main differentiator between each interference field is the difficulty of the sensor signatures, which is the evidence for this problem. The parameter that best reflects task difficulty is v, drift rate. τ is a constant that relates to hitting time, β controls the starting threshold, and a controls how much evidence is needed, but not relates to the type of evidence itself. On the other hand, drift rate, talks about how reliable the evidence is, which matches the differentiator in this problem the closet out of all the parameters.

OUTPUT

```
fit.summary()
```
✓ 0.4s

	Mean	MCSE	StdDev	MAD	5%	50%	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	-66.732800	0.084874	3.506870	3.463540	-72.919000	-66.360200	-61.657600	1744.83	2499.34	21.0456	1.003220
v1[1]	0.807089	0.002542	0.151348	0.154736	0.561780	0.804814	1.056280	3597.43	3317.88	43.3911	1.001440
v1[2]	0.714739	0.002976	0.197927	0.193229	0.389879	0.715680	1.043430	4475.40	3245.29	53.9810	1.000360
v1[3]	0.660573	0.003148	0.216081	0.220054	0.299146	0.661621	1.012120	4761.78	3456.73	57.4352	1.001160
v1[4]	0.457942	0.003063	0.202412	0.202689	0.121356	0.463862	0.782194	4431.85	2975.61	53.4557	1.000720
v1[5]	0.247054	0.003001	0.213327	0.216995	-0.112781	0.249677	0.599066	5056.49	3177.54	60.9899	0.999520
v2[1]	3.490960	0.004577	0.307988	0.312522	2.996010	3.486290	4.000530	4691.99	3208.42	56.5935	1.000220
v2[2]	3.725210	0.005266	0.398090	0.395417	3.057460	3.732270	4.370200	5820.00	3463.09	70.1991	0.999482
v2[3]	3.353610	0.005845	0.402507	0.404021	2.683410	3.359550	4.007960	4843.45	2896.88	58.4203	1.000780
v2[4]	3.289840	0.005225	0.377834	0.370114	2.679090	3.284500	3.911890	5282.76	3374.57	63.7192	0.999589
v2[5]	3.366330	0.005192	0.351299	0.352250	2.771790	3.367490	3.926270	4674.31	3440.30	56.3802	1.001600
a[1]	2.160050	0.002135	0.132161	0.128890	1.952930	2.150680	2.394460	3937.94	3191.21	47.4983	1.002050
a[2]	1.763320	0.001536	0.116697	0.118715	1.581700	1.758890	1.962110	5793.43	3174.84	69.8787	1.000330
a[3]	1.636800	0.001382	0.107182	0.105800	1.472940	1.632150	1.824560	6182.55	3206.37	74.5721	1.002020
a[4]	1.581120	0.001246	0.091720	0.091180	1.438130	1.575880	1.738130	5517.21	2784.98	66.5470	1.000030
a[5]	1.421280	0.001159	0.076048	0.074792	1.300050	1.418360	1.553260	4383.25	3217.49	52.8695	1.002170
beta[1]	0.411198	0.000700	0.039039	0.039759	0.346719	0.411340	0.473822	3176.73	2810.16	38.3168	1.002370
beta[2]	0.603732	0.000810	0.044496	0.045526	0.528441	0.605512	0.673434	3145.02	3013.00	37.9344	1.000490
beta[3]	0.648310	0.000729	0.041629	0.041544	0.577095	0.649874	0.713319	3314.52	2734.15	39.9787	1.002180
beta[4]	0.596986	0.000754	0.042576	0.042796	0.523755	0.598670	0.663124	3230.11	2589.03	38.9607	1.000020
beta[5]	0.464739	0.000611	0.036852	0.036476	0.402304	0.465121	0.524463	3659.35	2436.01	44.1380	1.000190
tau_raw[1]	0.750416	0.000918	0.051379	0.050208	0.659784	0.755784	0.824332	3292.37	3083.36	39.7116	1.001320
tau_raw[2]	0.899589	0.000470	0.027631	0.026789	0.849744	0.902385	0.940044	3411.62	3120.27	41.1500	1.000420
tau_raw[3]	0.926588	0.000359	0.022220	0.020808	0.885672	0.929557	0.957354	3948.01	3011.29	47.6197	1.000670
tau_raw[4]	0.895094	0.000500	0.028237	0.026827	0.843330	0.898286	0.935788	3251.75	3174.07	39.2216	0.999873
tau_raw[5]	0.840578	0.000601	0.037374	0.036967	0.773243	0.844166	0.895010	3859.13	3391.68	46.5477	1.000020
tau[1]	0.389915	0.000477	0.026697	0.026088	0.342823	0.392705	0.428322	3292.37	3083.36	39.7116	1.001320
tau[2]	0.382864	0.000200	0.011760	0.011401	0.361650	0.384054	0.400082	3411.62	3120.27	41.1500	1.000430
tau[3]	0.393984	0.000153	0.009448	0.008848	0.376587	0.395247	0.407066	3948.00	3011.29	47.6197	1.000670
tau[4]	0.403597	0.000226	0.012732	0.012096	0.380257	0.405036	0.421946	3251.75	3174.07	39.2216	0.999871
tau[5]	0.263100	0.000188	0.011698	0.011570	0.242024	0.264223	0.280137	3859.13	3391.68	46.5477	1.000020

ANALYSIS

We observe from the posterior estimates that the parameter that captures task difficulty is indeed v. From the posterior summaries, we observe that the estimates for v_1 are consistently smaller across all operators than the estimates for v_2. Specifically, the values for v_1 are in the range of $[.25, .81]$, while the values for v_2 are around $[3.29, 3.73]$. From the problem description, high-interference field represents more difficult and noisier environments, which corresponds with low values of v. On the other hand, low-interference field represents more easy and clearer environments, which corresponds with high values of v. So, we conclude that Condition 1 corresponds to high-interference field and Condition 2 corresponds to low-interference field. Furthermore, from the summary, we observe that $\hat{R}$ values are all close to

1, which means that there is good convergence across the chains. In terms of efficiency diagnostics, the effective sample size (ESS) is rather large across all parameters, which suggests that the Markov chains are producing nearly independent samples and exploring the posterior distribution efficiently. Thus, overall the MCMC sampling is both quite efficient and well-converged. Hence, our conclusion of Condition 1 is the high-interference field is well supported by our DDM model.